

NYCU Introduction to Machine Learning, Homework 1

[112550069], [謝佑哲]

Part. 1, Coding (60%):

(10%) Linear Regression Model - Closed-form Solution

1. (10%) Show the weights and intercepts of your linear model.

```
2024-10-01 16:01:30.981 | INFO | __main__:main:109 - LR_CF.weights=array([2.8491883 , 1.0188675 , 0.48562739, 0.1937254
]), LR_CF.intercept=-33.8223
```

(40%) Linear Regression Model - Gradient Descent Solution

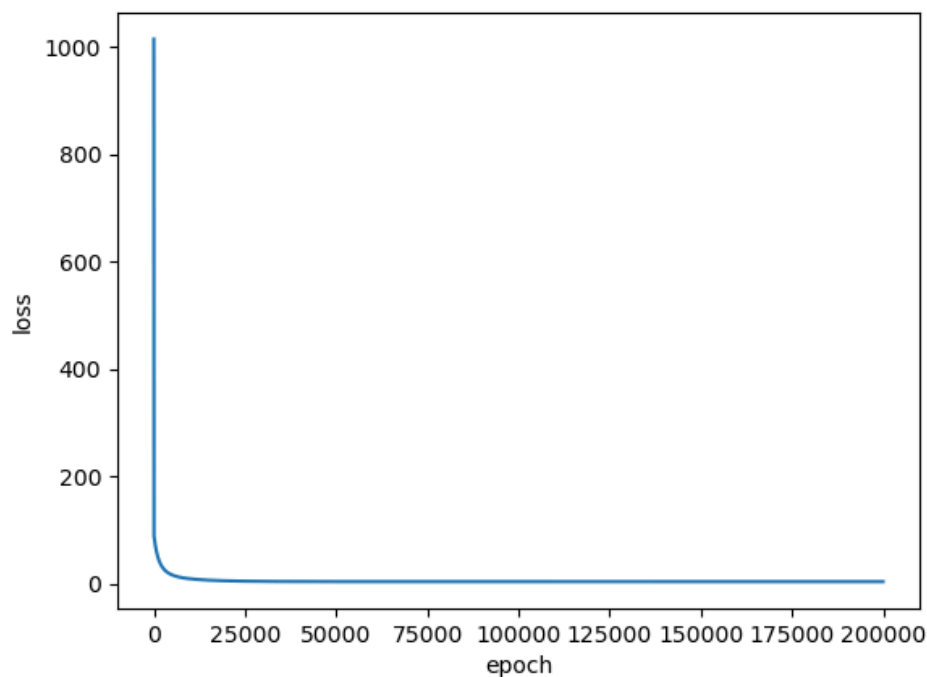
2. (10%)
 - Show the hyperparameters of your setting (e.g., learning rate, number of epochs, batch size, etc.).
 - Show the weights and intercepts of your linear model.

I didn't change the batch size.

```
losses = LR_GD.fit(train_x, train_y, learning_rate=2e-1, epochs=200000)

2024-10-01 16:15:00.512 | INFO | __main__:main:114 - LR_GD.weights=array([2.84918571, 1.01886683, 0.48561592, 0.1937236
4]), LR_GD.intercept=-33.8221
```

3. (10%) Plot the learning curve. (x-axis=epoch, y-axis=training loss)



In this part, I provide loss in 0, 10000, 20000 epochs for clearer realization:

```

2024-10-01 19:06:54.829 | INFO | __main__:fit:49 - EPOCH 0, loss=1014.900375
2024-10-01 19:06:56.171 | INFO | __main__:fit:49 - EPOCH 10000, loss=9.474801363875667
2024-10-01 19:06:57.461 | INFO | __main__:fit:49 - EPOCH 20000, loss=5.736741610863818

```

- (20%) Show your MSE.cf, MSE.gd, and error rate between your closed-form solution and the gradient descent solution.

```

2024-10-01 16:15:00.515 | INFO | __main__:main:123 - Mean prediction difference: 0.0000
2024-10-01 16:15:00.515 | INFO | __main__:main:128 - mse_cf=4.1997, mse_gd=4.1997. Difference: 0.000%

```

(10%) Code Check and Verification

- (10%) Lint the code and show the PyTest results.

```
(AIHomework) C:\Users\money\桌面\學校課業資料夾\大二上\ML\NYCU-Intro-ML\hw1>flake8 main.py
```

```

(AIHomework) C:\Users\money\桌面\學校課業資料夾\大二上\ML\NYCU-Intro-ML\hw1>pytest ./test_main.py -s
===== test session starts =====
platform win32 -- Python 3.8.18, pytest-7.4.4, pluggy-1.0.0
rootdir: C:\Users\money\桌面\學校課業資料夾\大二上\ML\NYCU-Intro-ML\hw1
plugins: anyio-4.4.0
collected 2 items

test_main.py 2024-10-01 16:06:32.442 | INFO | test_main:test_regression_cf:27 - model.weights=array([[3.]]), model.intercept=array([4.])
.2024-10-01 16:06:32.443 | INFO | main:fit:49 - EPOCH 0, loss=13611.693602693605
2024-10-01 16:06:32.751 | INFO | main:fit:49 - EPOCH 10000, loss=0.8163440081967882
2024-10-01 16:06:33.040 | INFO | main:fit:49 - EPOCH 20000, loss=0.07404639330208403
2024-10-01 16:06:33.305 | INFO | main:fit:49 - EPOCH 30000, loss=0.006716369944527903
2024-10-01 16:06:33.595 | INFO | main:fit:49 - EPOCH 40000, loss=0.0006092075956718253
2024-10-01 16:06:33.878 | INFO | main:fit:49 - EPOCH 50000, loss=5.525810783036669e-05
2024-10-01 16:06:34.167 | INFO | main:fit:49 - EPOCH 60000, loss=5.012180581287499e-06
2024-10-01 16:06:34.474 | INFO | test_main:test_regression_gd:39 - model.weights=array([3.]), model.intercept=3.9993257379683977
.
===== 2 passed in 3.59s =====

```

Part. 2, Questions (40%):

- (10%) How does the presence of outliers affect the performance of a linear regression model? How should outliers be handled? List at least two methods.

The presence of outliers will cause wrong gradient direction (influence MSE and then impact the gradient further) which will produce errors in the parameters of the linear regression model. Especially, when the linear model is close to the best solution, the partition of error (MSE) will be larger, which is to influence server.

As for how to handle the outlier, I provide 3 methods. First, we can use the RANSAC method. This method is usually used in CV fields for improving auto stitching tasks (eg: SIFT). It assumes different data points as inlier points and computes the model difference when adding new points, classifying those points to outliers or inliers by some threshold. The solution of most inliers is the best solution that excludes outliers.

Second, we can use MSE to exclude outliers. When the linear model is trained for enough iterations, it should be representative enough of data of linear distribution. At this time we can find the points with large MSE, it can be regarded as outliers

because using MSE means we assume the point will happen with the gaussian probability distribution with the line as mean. In other words, the probability of points with large MSE being inliers is very very little.

Third, we can use some clustering algorithms like DBSCAN. With these methods, we can cluster inliers and find the points not in the cluster. The points' density with the cluster is too low that we can view them as outliers.

2. (15%) How do different values of learning rate (too large, too small...) affect the convergence of optimization? Please explain in detail.

With too large learning rate, it may make the model hard to converge to the global minimum of loss because the step of gradient is too large which will cause the parameters cannot be tuned finely. When the model's parameters capacity is larger, the fine difference will influence more.

With too small learning rate, it may make the model converge in local minimum. Assume there is a loss model's projection on 3D views, the loss may converge in a pothole that is much higher than global maximum. In other words, small learning rate may lead to loss is unable to cross a local barrier and go to the lower place.

3. (15%)
- What is the prior, likelihood, and posterior in Bayesian linear regression. [Explain the concept in detail rather than writing out the mathematical formula.]
 - What is the difference between Maximum Likelihood Estimation (MLE) and Maximum A Posteriori Estimation (MAP)? (Analyze the assumptions and the results.)

1.

In my realization, before we discuss the Bayesian model, we need to define what are events and evidences (though they are both called events in probability). When we discuss the Bayesian model, the events are usually what we want to infer (often denoted by Y) and the evidence are usually what we have to infer the events (often denoted by X).

With above definitions, prior probability $P(Y)$ is the events' probability distribution we assume. Our updating object is also prior probability $P(Y)$. The likelihood $P(X|Y)$ means the likelihood of evidence X when events Y happen. In other words, it determines how we should update the prior probability $P(Y)$. With large $P(X|Y)$ means the evidence X may be highly relative to events Y . When we get such evidence X , we will increase events Y 's probability in our inference. The posterior $P(Y|X)$ is the probability we infer for Y with evidence. It's the probability after we consider new evidence. That is why it is named posterior probability.

In conclusion, we assume a prior probability $P(Y)$ with old evidence. Then, we consider evidence X , with likelihood $P(X|Y)$ we can infer new posterior $P(Y|X)$. After that, the evidence X will be old evidence and $P(Y|X)$ will be our new prior probability.

2.

The most significant difference between MLE and MAP is whether we assume a prior distribution. For MLE solution, we trivially want to make $P(t|x, w, \beta)$ larger, which means the target points (data point in dataset) have higher probability happening in our estimated distribution. This way will lead to a result that our loss function of w is $\sum_{n=1}^N \{y(x, w) - t\}^2$ (inferred from $\log(P(t|x, w, \beta))$ and exclude elements without w).

Different from MLE, MAP with a assumption that we have prior distribution and our maximized object is posterior distribution $P(w|x, t, \alpha, \beta)$ (the posterior distribution consider w as objective event and other elements as evidence). Maximizing posterior probability means we want to make our weight w is the most proper value in condition of input x and target t etc. When the posterior probability is larger, the weight will be more suitable to fit the data. Then, with Bayesian theorem, we can optimize (likelihood * prior) to optimize posterior probability because they are proportional. As a result of MAP, the loss function of w is $\sum_{n=1}^N \{y(x, w) - t\}^2 + \alpha/2 * w^T w$, it moreover considers w^2 which is like what regularization does. It will make the weight limit and deal with the problem of overfitting.